



Pertemuan
[11]

REKAYASA PERANGKAT LUNAK

Teknik Development

Nurul Tiara Kadir, S.Kom., M.Eng.

Informatika
2026

APA ITU TEKNIK DEVELOPMENT?

Definisi

Teknik Development (atau **Software Development Techniques**) adalah serangkaian metode, pendekatan, dan praktik yang digunakan oleh developer untuk membangun perangkat lunak secara efisien, berkualitas, dan dapat dipelihara.

Teknik ini mencakup seluruh aspek siklus hidup pengembangan, dari metodologi proyek hingga penulisan kode yang bersih.

Ruang Lingkup

- Metodologi Pengembangan
Waterfall, Agile, Scrum, Kanban
- Pola Perancangan
Design patterns & SOLID principles
- Manajemen Kode
Version control, code review, standards
- Pengujian & Refactoring
Unit test, TDD, continuous improvement

Fondasi Teknik Development

Teknik Development adalah serangkaian metode, pendekatan, dan praktik yang digunakan oleh developer untuk membangun perangkat lunak secara efisien, berkualitas, dan dapat dipelihara.

Lebih dari sekadar menulis kode, teknik ini memastikan kelancaran kolaborasi tim dan kesiapan kode untuk fase produksi.



Metodologi Pengembangan

Cara tim mengatur ritme kerja dan berkolaborasi (Agile, Scrum).



Praktik Teknis

Standar penulisan kode, perancangan, dan pengujian (Design Patterns, S.O.L.I.D, TDD).



Ekosistem Tools

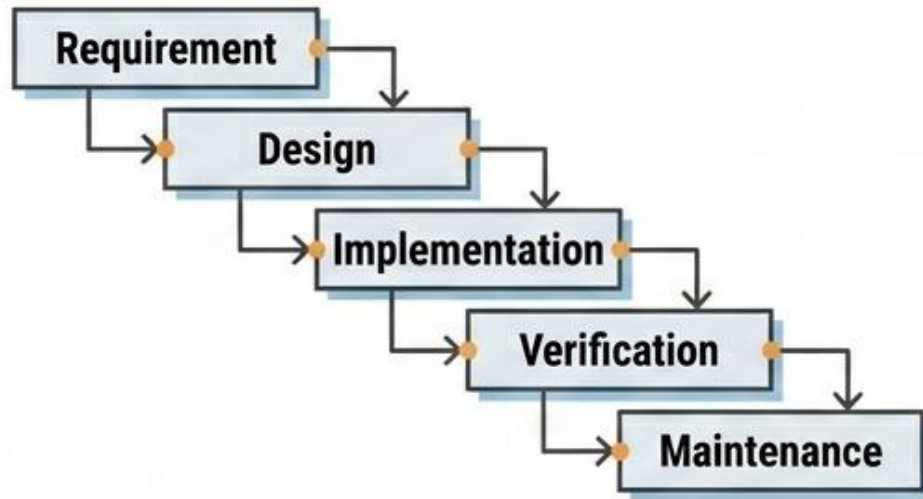
Perangkat lunak yang mengotomatisasi dan menjaga kualitas kode.



Metodologi Pengembangan Perangkat Lunak

Metodologi adalah kerangka kerja yang menentukan **bagaimana tim merencanakan, mengeksekusi, dan mengelola** sebuah proyek perangkat lunak. Setiap metodologi memiliki filosofi, kelebihan, dan konteks penggunaan yang berbeda.

Metodologi: Tradisional vs. Modern



Waterfall (Tradisional)

Fase berurutan dan kaku. Setiap fase harus selesai sebelum lanjut.

Karakteristik: Dokumentasi sangat lengkap, cocok untuk regulasi ketat (pemerintahan/perbankan).



Agile (Modern)

Filosofi pengembangan iteratif dan inkremental.

Fokus pada adaptasi perubahan dan pengiriman nilai bertahap.

4 Nilai Utama (Agile Manifesto):

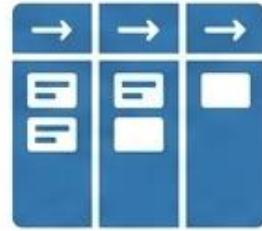
- Individu & interaksi > Proses & alat
- Software berfungsi > Dokumentasi lengkap
- Kolaborasi pelanggan > Negosiasi kontrak
- Merespons perubahan > Mengikuti rencana

Framework Agile Terpopuler



Scrum (Iterasi Pendek)

- Membagi pekerjaan menjadi Sprint (1–4 minggu).
- Peran: Product Owner, Scrum Master, Development Team.
- Event Utama: Planning, Daily Stand-up (15 menit), Review, Retrospective.



Kanban (Aliran Kontinu)

- Pendekatan visual berbasis papan (To Do → In Progress → Review → Done).
- Kunci utama: Membatasi Work-In-Progress (WIP).
- Sangat cocok untuk maintenance atau tim dengan prioritas dinamis.



Extreme Programming / XP (Kualitas Teknis)

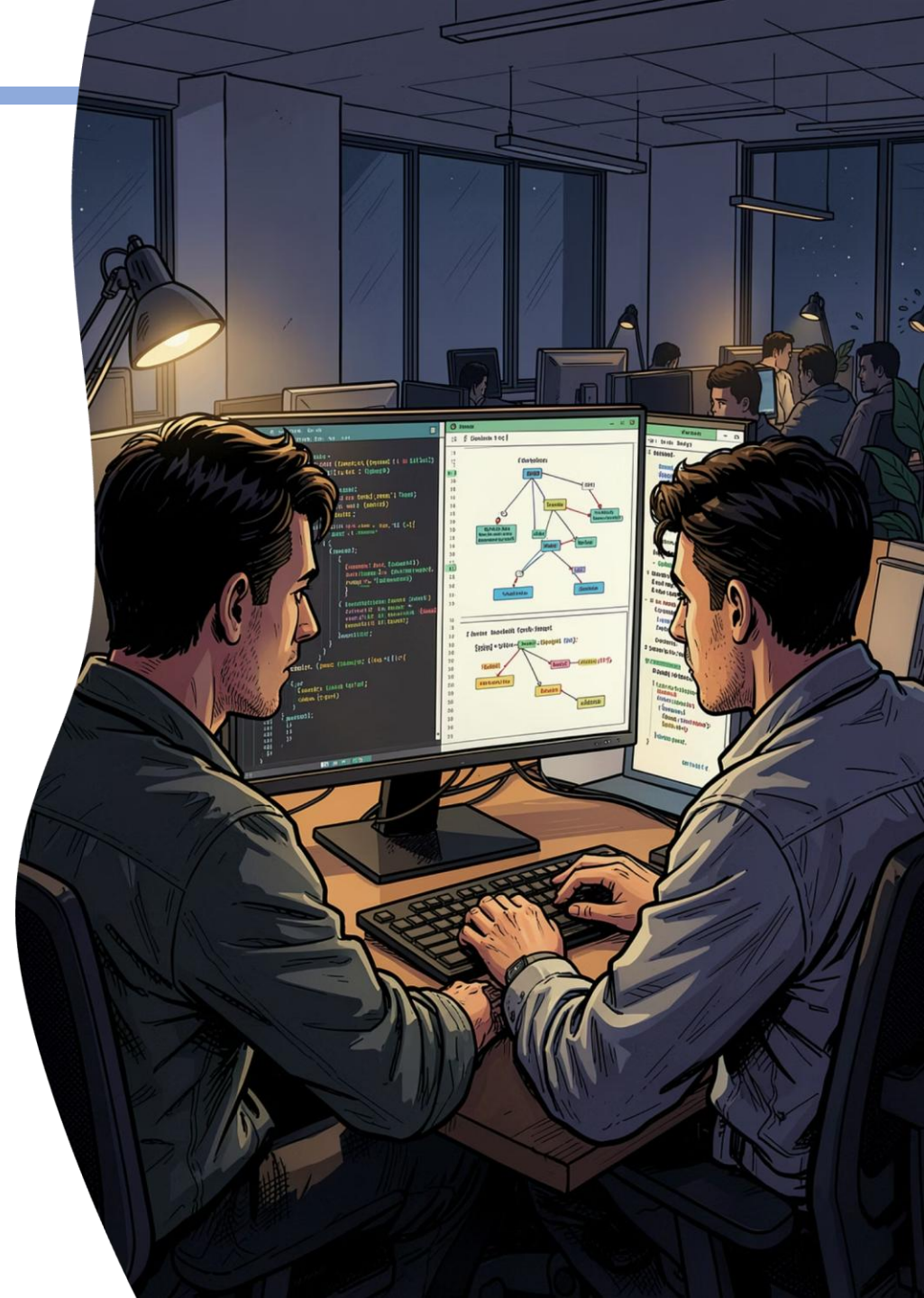
- Fokus pada praktik pemrograman yang ekstrem.
- Menerapkan Test-Driven Development (TDD).
- Menggunakan Pair Programming (dua developer, satu layar).

Matriks Komparasi Metodologi

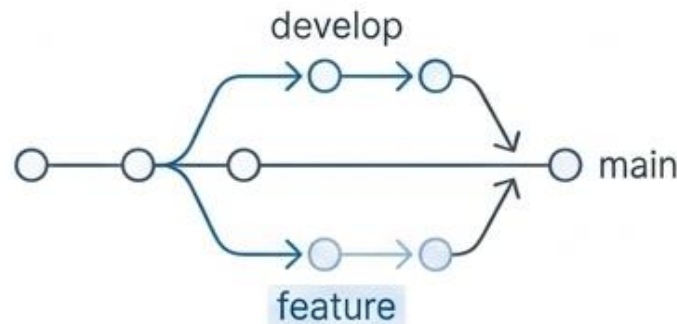
Aspek	Waterfall	Scrum	Kanban
Iterasi	Tidak ada	Sprint 1–4 minggu	Continuous flow
Perubahan	Sangat sulit beradaptasi	Dapat diubah antar sprint	Dapat diubah kapan saja
Dokumentasi	Sangat lengkap & kaku	Secukupnya	Minimal
Kecocokan	Kebutuhan proyek stabil	Pengembangan produk baru	Maintenance & Support
Tingkat Risiko	Tinggi (bug terlambat dideteksi)	Rendah	Rendah

Praktik Teknis Development

Di luar metodologi proyek, ada sekumpulan **praktik teknis harian** yang membedakan developer biasa dengan developer profesional — dari version control hingga penerapan prinsip desain yang solid.



Praktik Teknis: Manajemen & Standar Kode



Version Control (Git)

- Sistem pencatat perubahan kode (Git Flow, GitHub Flow).
- Conventional Commits: Standar pesan commit:
 - feat: fitur baru
 - fix: perbaikan bug
 - docs: perubahan dokumen

Code Review

- Pemeriksaan kode oleh rekan tim sebelum di-merge.
- Tujuan: Mencegah technical debt, knowledge sharing, dan menemukan bug lebih awal.

○○○

Coding Standard & Style Guide

- Mengikuti konvensi seragam berdasar bahasa pemrograman untuk menjaga keterbacaan:
 - ESLint / Prettier (JavaScript)
 - PEP 8 (Python)
 - Google Java Style (Java)

VERSION CONTROL DENGAN GIT

Version control adalah sistem yang mencatat perubahan kode dari waktu ke waktu. Git adalah standar de facto saat ini.

Workflow umum yang sering dipakai:

- **Git Flow:** branch master, develop, feature, release, hotfix. Cocok untuk rilis terjadwal.
- **GitHub Flow:** branch main + feature branches. Sederhana dan cocok untuk continuous deployment.
- **Trunk-Based Development:** semua developer commit ke main dengan rilis singkat. Cocok untuk tim matang.

Pola Perancangan (Design Patterns)

Solusi terbukti untuk masalah perancangan struktur kode yang sering berulang.



MVC / MVVM

Memisahkan secara tegas logika tampilan (View), data (Model), dan kontrol (Controller).



Repository

Abstraksi akses data agar logika bisnis tidak bergantung langsung pada database.



Singleton

Memastikan hanya ada satu instance global yang berjalan (misal: koneksi database).



Factory

Menciptakan objek tanpa perlu mengetahui kelas konkretnya secara spesifik.



Observer

Satu objek otomatis memberi tahu banyak objek lain saat state-nya berubah.

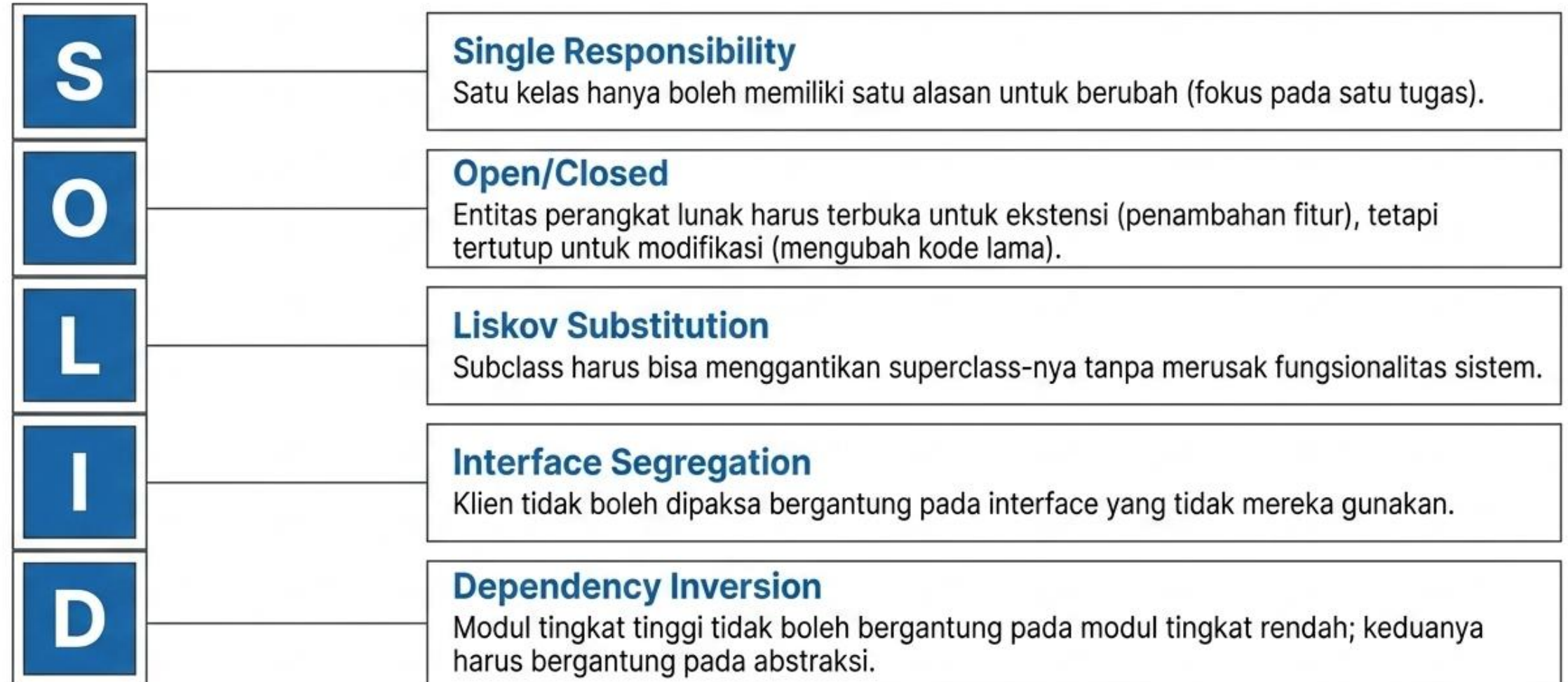


Strategy

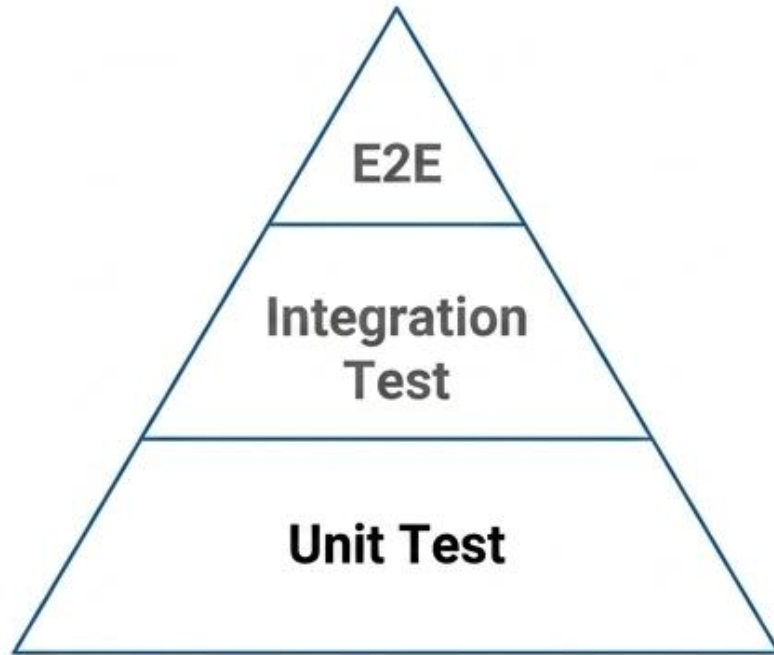
Menukar algoritma secara dinamis saat runtime (sangat krusial untuk pipeline AI/ML).

Prinsip S.O.L.I.D

Fondasi perancangan perangkat lunak berbasis objek yang bersih dan maintainable.

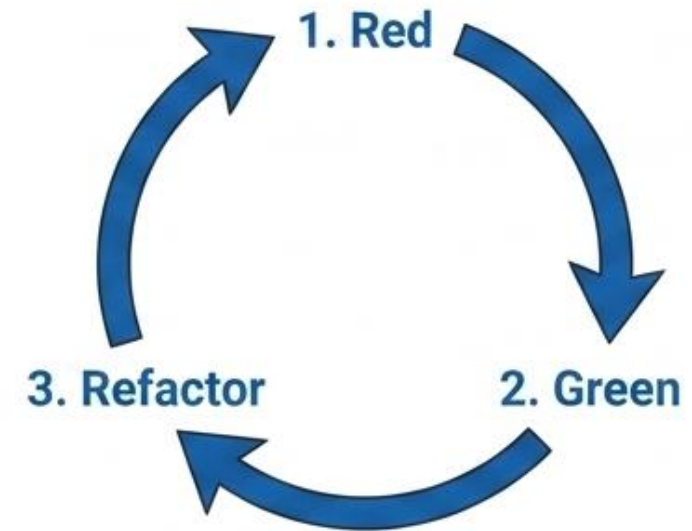


Pengujian (Testing) & Refactoring



Pengujian Selama Development

- **Unit Test:** Menguji satu fungsi/kelas secara terisolasi.
- **Integration Test:** Menguji interaksi antar modul (misal: API ke Database).
- **End-to-End (E2E):** Menguji alur sistem utuh dari sudut pandang pengguna.



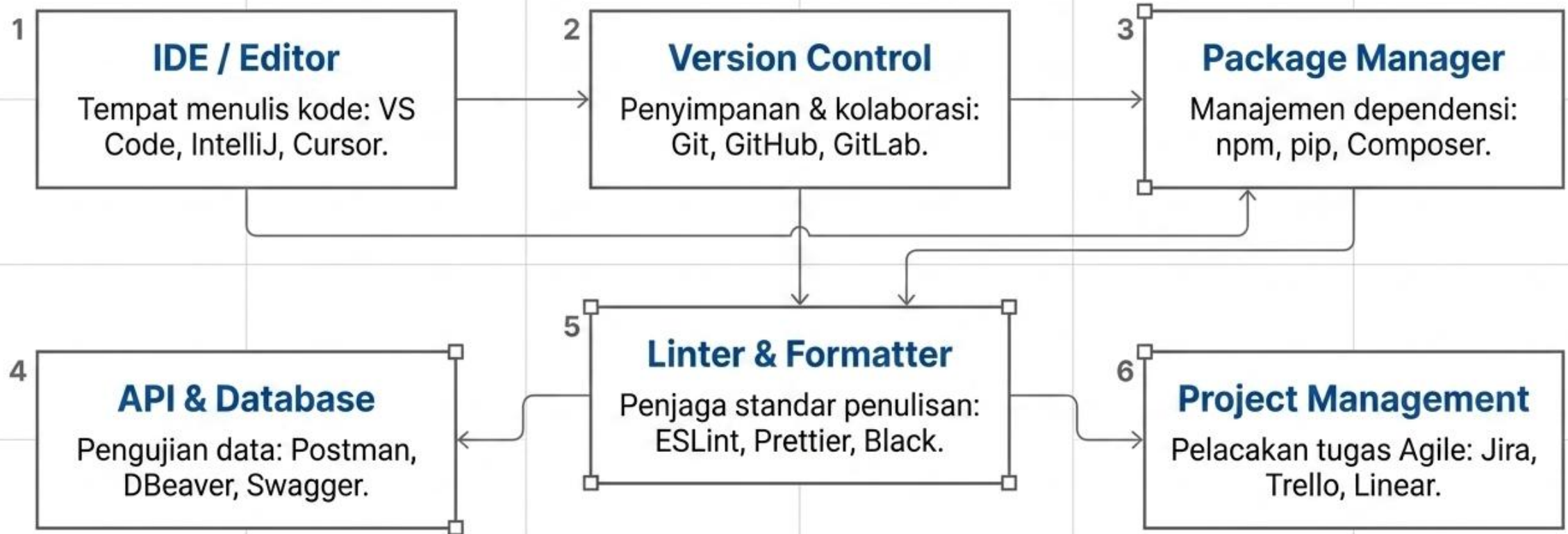
Test-Driven Development (TDD)

1. Tulis test yang gagal (Red).
2. Tulis kode minimal agar lulus (Green).
3. **Refactoring:** Perbaiki struktur internal kode tanpa mengubah perilakunya.

Aturan Emas: Tinggalkan kode dalam keadaan lebih bersih dari sebelumnya.

Ekosistem Tools Pendukung Industri

Perangkat lunak krusial untuk otomatisasi dan manajemen proyek sehari-hari.



Tools Pendukung Development

Ekosistem tools yang tepat meningkatkan produktivitas dan kualitas kode secara signifikan. Berikut adalah kategori tools utama yang wajib dikuasai oleh developer profesional:



IDE & Version Control

IDE: VS Code, IntelliJ IDEA, Android Studio, Cursor

VCS: Git, GitHub, GitLab, Bitbucket



Package Manager & API Tools

Package: npm/pnpm, pip, Gradle, Pub

API: Postman, Insomnia, Swagger/OpenAPI



Database & Linter

DB Tools: DBeaver, TablePlus, MongoDB Compass

Linter: ESLint, Prettier, Black, Checkstyle



Project Management

Jira, Trello, ClickUp, Linear, Notion — kelola sprint, backlog, dan kolaborasi tim secara terpusat dan transparan.

👉 Menguasai kombinasi tools yang tepat — bukan semua tools — adalah kunci produktivitas developer yang sesungguhnya.

Sintesis: Kesatuan Implementasi Perangkat Lunak



Keberhasilan fase implementasi bukan sekadar dari kemampuan coding, melainkan perpaduan presisi dari:

1. **Metodologi** yang merespons dinamika proyek secara tepat.
2. **Praktik Teknis** (S.O.L.I.D, Patterns, TDD) yang menjamin arsitektur yang scalable.
3. **Ekosistem Tools** yang mengotomatisasi pekerjaan dan menjaga standar.

Hasil Akhir: Rangkaian ini menghasilkan source code yang andal, bersih, dan siap untuk diserahkan menuju fase **Deployment** dan **validasi Black Box / Usability Testing**.

ANY QUESTIONS?



